

Autoscaling groups of instances



Managed instance groups (MIGs) (/compute/docs/instance-groups) offer autoscaling capabilities that let you automatically add or delete virtual machine (VM) instances from a MIG based on increases or decreases in load. Autoscaling helps your apps gracefully handle increases in traffic and reduce costs when the need for resources is lower. You define the autoscaling policy (#autoscaling_policy) and the autoscaler performs automatic scaling based on the measured load and the options you configure.

Autoscaling works by adding more VMs to your MIG when there is more load (scaling out), and deleting VMs when the need for VMs is lowered (scaling in).

Prerequisites

The autoscaler uses the Compute Engine Service Agent (/compute/docs/access/service-accounts#compute_engine_service_account) to add and remove instances in the group. Google Cloud automatically creates this service account, as well as its IAM policy binding to the Compute Engine Service Agent role, when the Compute Engine API is enabled (<https://console.cloud.google.com/apis/api/compute.googleapis.com/overview>).

If your project is missing this account—for instance, if you have removed it—you can add it manually:

Consolegcloud (#gcloud)
(#console)

1. In the Google Cloud console, go to the **IAM** page.

Go to IAM (<https://console.cloud.google.com/iam-admin/iam>)

2. Click **Grant Access**.
3. In the **New principals** field, enter `service-PROJECT_NUMBER@compute-system.iam.gserviceaccount.com`.
4. Select the **Compute Engine Service Agent** role.
5. Click **Save**.

Fundamentals

Autoscaling uses the following fundamental concepts and services.

Managed instance groups

Autoscaling is a feature of [managed instance groups \(MIGs\)](/compute/docs/instance-groups) (/compute/docs/instance-groups). A managed instance group is a collection of virtual machine (VM) instances that are created from a common [instance template](/compute/docs/instance-templates) (/compute/docs/instance-templates). An autoscaler adds or deletes instances from a managed instance group based on the group's autoscaling policy. Although Compute Engine has both managed and unmanaged instance groups, only managed instance groups can be used with an autoscaler.

To understand the difference between a managed instance group and an unmanaged instance group, see [Instance groups](/compute/docs/instance-groups) (/compute/docs/instance-groups).

To learn how to create a managed instance group, see [Creating MIGs](/compute/docs/instance-groups/creating-groups-of-managed-instances) (/compute/docs/instance-groups/creating-groups-of-managed-instances).

Autoscaling policy

When you define an autoscaling policy for your group, you specify one or more signals that the autoscaler uses to scale the group. When you set multiple signals in a policy, the autoscaler calculates the recommended number of VMs for each signal and sets your group's recommended size to the largest number.

An autoscaling policy must always have at least one scaling signal. When you turn on autoscaling in a MIG, by default, the autoscaler adds a CPU utilization signal. You can edit this default signal, or remove and add other signals in the policy.

The following sections provide an overview of signals based on target utilization metrics and signals based on schedules.

Target utilization metrics

You can autoscale based on one or more of the following metrics that reflect the load of the instance group:

- Average CPU utilization

- HTTP load balancing serving capacity
- Cloud Monitoring metrics

The autoscaler continuously collects usage information based on the selected utilization metric, compares actual utilization to your desired target utilization, and uses this information to determine whether the group needs to remove instances (scale in) or add instances (scale out).

The target utilization level is the level at which you want to maintain your virtual machine (VM) instances. For example, if you scale based on CPU utilization, you can set your target utilization level at 75% and the autoscaler will maintain the CPU utilization of the specified group of instances at or close to 75%. The utilization level for each metric is interpreted differently based on the autoscaling policy.

If you autoscale based on any of the following, then your MIG cannot scale in to zero instances:

- Average CPU utilization
- HTTP load balancing serving capacity
- Monitoring metrics that come from each instance in the MIG

However, you can use other Monitoring metrics when scaling in to zero instances, provided you set the minimum number of instances (`autoscalingPolicy.minNumReplicas`) to 0.

For more information about scaling based on target utilization metrics, see the following pages:

- [Scaling based on CPU utilization](/compute/docs/autoscaler/scaling-cpu) (/compute/docs/autoscaler/scaling-cpu)
- [Scaling based on load balancing serving capacity](/compute/docs/autoscaler/scaling-load-balancing)
(/compute/docs/autoscaler/scaling-load-balancing)
- [Scaling based on Cloud Monitoring metrics](/compute/docs/autoscaler/scaling-stackdriver-monitoring-metrics)
(/compute/docs/autoscaler/scaling-stackdriver-monitoring-metrics)

Schedules

You can use schedule-based autoscaling to allocate capacity for anticipated loads. You can have up to 128 scaling schedules per instance group. For each scaling schedule, specify the

following:

- **Capacity:** minimum required VM instances
- **Schedule:** start time, duration, and recurrence (for example, once, daily, weekly, or monthly)

Each scaling schedule is active from its start time and for the configured duration. During this time, the autoscaler scales the group to have at least as many instances as defined by the scaling schedule.

When using schedules, your MIG can scale in to zero instances if all of the following conditions are met:

- The minimum number of instances (`autoscalingPolicy.minNumReplicas`) is set to 0.
- The autoscaling policy doesn't contain any active schedules.
- The autoscaling policy doesn't contain signals based on target utilization metrics (`#target_utilization`) that prevent scaling in to zero instances.

For more information, see [Scaling based on schedules](/compute/docs/autoscaler/scaling-schedules) (/compute/docs/autoscaler/scaling-schedules).

Initialization period

The initialization period, formerly known as cool down period, is the duration it takes for applications to initialize on your VM instances. While an application is initializing on an instance, the instance's usage data might not reflect normal circumstances. So the autoscaler uses the initialization period for scaling decisions in the following ways:

- For scale-in decisions, the autoscaler considers usage data from all instances, even an instance that is still within its initialization period. The autoscaler recommends to remove instances if the average utilization from all instances is less than the target utilization.
- For scale-out decisions, the autoscaler ignores usage data from instances that are still in their initialization period.
- If you enable predictive mode (`#predictive_mode`), the initialization period informs the predictive autoscaler to scale out further in advance of anticipated load, so that applications are initialized when the load arrives. For example, if you set the initialization

period to 300 seconds, then predictive autoscaler creates VMs 5 minutes ahead of forecasted load.

By default, the initialization period is 60 seconds. Actual initialization times vary because of numerous factors. We recommend that you test how long your application takes to initialize. To do this, create an instance and time the startup process from when the instance becomes **RUNNING** (/compute/docs/instance-groups/getting-info-about-migs#verify_instances) until the application is ready.

If you set a initialization period value that is significantly longer than the time it takes for an instance to initialize, then your autoscaler might ignore legitimate utilization data, and it might underestimate the required size of your group, causing a delay in scaling out (/compute/docs/autoscaler/understanding-autoscaler-decisions#delays_in_scaling_out).

Stabilization period

Autoscaling signals like CPU utilization are not very stable and can change rapidly. As the load goes up and down, the autoscaler needs to stabilize the signal to avoid continuous VM deletion and creation. The autoscaler stabilizes a signal by keeping sufficient VM capacity in order to serve the peak load that is observed during the *stabilization period*.

The stabilization period is used only for scale-in decisions when the autoscaler has to delete VMs. When the load goes down, the autoscaler does not delete VMs immediately. The autoscaler keeps monitoring capacity needed for the duration of the stabilization period and deletes VMs only when there is sufficient capacity to meet the peak load. This might appear as a delay in scaling in.

The default stabilization period is 10 minutes. You can configure this period to use a value between 0 seconds and 3600 seconds (one hour). Setting the stabilization period to 0 seconds means that the autoscaler can scale in immediately when capacity is no longer needed, without any delay.

For more information on how to set the stabilization period and the best practices, see [Configure stabilization period](#) (/compute/docs/autoscaler/managing-autoscalers#configure_stabilization_period).

Autoscaling mode

If you need to investigate or configure your group without interference from autoscaler operations, you can temporarily [turn off or restrict autoscaling activities](#) (/compute/docs/autoscaler/managing-autoscalers#turn_off_or_restrict_an_autoscaler). The autoscaler's configuration persists while it is turned off or restricted, and all autoscaling activities resume when you turn it on again or lift the restriction.

Predictive autoscaling

If you enable predictive autoscaling to optimize your MIG for availability, the autoscaler forecasts future load based on historical data and scales out a MIG in advance of predicted load, so that new instances are ready to serve when the load arrives.

Predictive autoscaling works best if your workload meets the following criteria:

- Your application takes a long time to initialize—for example, if you configure a [initialization period](#) (#cool_down_period) of more than 2 minutes.
- Your workload varies predictably with daily or weekly cycles.

For more information, see [Scaling based on predictions](#) (/compute/docs/autoscaler/predictive-autoscaling).

Scale-in controls

If your workloads take many minutes to initialize (for example, due to lengthy installation tasks), you can reduce the risk of response latency caused by abrupt scale-in events by [configuring scale-in controls](#) (/compute/docs/autoscaler/managing-autoscalers#configuring_scale-in_controls). Specifically, if you expect load spikes to follow soon after declines, you can limit the scale-in rate to prevent autoscaling from reducing a MIG's size by more VM instances than your workload can tolerate.

You don't have to configure scale-in controls if your application initializes quickly enough to pick up load spikes on scale out.

To configure scale-in controls, set the following properties in your autoscaling policy.

- **Maximum allowed reduction.** The number of VM instances that your workload can afford to lose (from its peak size) within the specified trailing time window. Use this parameter to limit how much your group can be scaled in so that you can still serve a likely load

spike until more instances start serving. The smaller you set the maximum allowed reduction, the longer it takes for your group to scale in.

- **Trailing time window.** The history within which the autoscaler monitors the peak size required by your workload. The autoscaler will not resize below the maximum allowed reduction subtracted from the peak size observed in this period. You can use this parameter to define how long the autoscaler should wait before removing instances, as defined by the maximum allowed reduction. With a longer trailing time window, the autoscaler considers more historical peaks, making scale-in more conservative and stable.

For more information, see [Configuring scale-in controls](#)

([/compute/docs/autoscaler/managing-autoscalers#configuring_scale-in_controls](#)) and [Understanding autoscaler decisions](#) ([/compute/docs/autoscaler/understanding-autoscaler-decisions#scale-in_controls](#))

Recommended size

The recommended group size is the autoscaler's recommended number of VMs that the managed instance group should maintain, based on peak load observed during the last 10 minutes. These last 10 minutes are referred to as the [stabilization period](#)

([/compute/docs/autoscaler/understanding-autoscaler-decisions#delays_in_scaling_in](#)). The recommended size is recalculated constantly. If you set an autoscaling policy with scale-in controls, then the recommended size is constrained by your scale-in controls.

For more information, see [How autoscalers calculate recommended size and affect target size](#)

([/compute/docs/autoscaler/understanding-autoscaler-decisions#how_autoscalers_calculate_recommended_size_and_affect_target_size](#))

Limitations

- You cannot use autoscaling with the following instance groups, which don't allow the autoscaler to create or delete VMs according to demand:
 - [Unmanaged instance groups](#)
([/compute/docs/instance-groups#unmanaged_instance_groups](#))

- MIGs with stateful configuration
(/compute/docs/instance-groups/stateful-migs#stateful_configuration)
- MIGs with VM repairs turned off
(/compute/docs/instance-groups/about-repair#turn_off_VM_repairs)
- You cannot create VM instances with specific names
(/compute/docs/instance-groups/add-remove-vms-in-mig#creating_instances_with_specific_names_in_migs)
while autoscaling is turned on.
- Do not use Compute Engine autoscaling with MIGs that are owned by Google Kubernetes Engine. For Google Kubernetes Engine groups, use cluster autoscaling
(/kubernetes-engine/docs/cluster-autoscaler) instead. If you're not sure whether a MIG is part of a GKE cluster, look for the `gke` prefix in the MIG name. For example, `gke-test-1-3-default-pool-eadj9ah`.

What happens during autohealing

Autoscaling works independently from autohealing

(/compute/docs/instance-groups/autohealing-instances-in-migs). If you configure autohealing for your group and an instance fails the health check, the MIG attempts to recreate the instance. While an instance is being recreated by the MIG, the number of running instances in the group might be lower than the minimum number of instances specified for the group (`autoscalingPolicy.minNumReplicas`).

Pricing

There is no additional charge for configuring an autoscaling policy. Autoscaler dynamically adds or deletes VM instances, so you are charged only for the resources that your MIG uses. You can control resource cost by configuring the minimum and maximum number of instances in the autoscaling policy. For Compute Engine pricing information, see Pricing
(https://cloud.google.com/compute/all-pricing).

What's next

1. Learn how autoscaling works in a regional MIG

(/compute/docs/instance-groups/regional-migs#autoscaling_a_regional_mig).

2. If you don't have an existing MIG, review how to create a managed instance group

(/compute/docs/instance-groups/creating-groups-of-managed-instances#create_managed_group).

3. Create an autoscaler that scales on:

- CPU utilization (/compute/docs/autoscaler/scaling-cpu)
- Load balancing serving capacity (/compute/docs/autoscaler/scaling-load-balancing)
- Cloud Monitoring metrics
(/compute/docs/autoscaler/scaling-stackdriver-monitoring-metrics)
- Schedules (/compute/docs/autoscaler/scaling-schedules)

4. Manage your autoscaler (/compute/docs/autoscaler/managing-autoscalers), for example, to get information about an autoscaler, to control the scale-in rate, or to temporarily restrict autoscaling.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the Apache 2.0 License (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the Google Developers Site Policies (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-08-11 UTC.